

INTERACTION DESIGN CASE STUDY

Creative Operations Platform

From a personal portfolio to a unified, enterprise-pattern product prototype.

PRODUCT PROMISE

One workspace to publish, distribute, understand, and evolve creative work.



Hero image placeholder • Recommended crop: 16:7

ROLE Product strategy • Interaction design • Visual design • Full-stack prototyping

STAGE Working single-operator platform • Enterprise validation roadmap defined

CORE STACK Astro • React • TypeScript • Supabase • Cloudflare R2 • Vercel • Resend

SCOPE Public site • Admin workspace • AI tools • Media pipeline • Analytics

The project began with a personal need: manage a growing body of work and repeated enquiries without switching between disconnected tools. The same workflow problem appears across independent creators, studios, and small teams.

01 • OPPORTUNITY

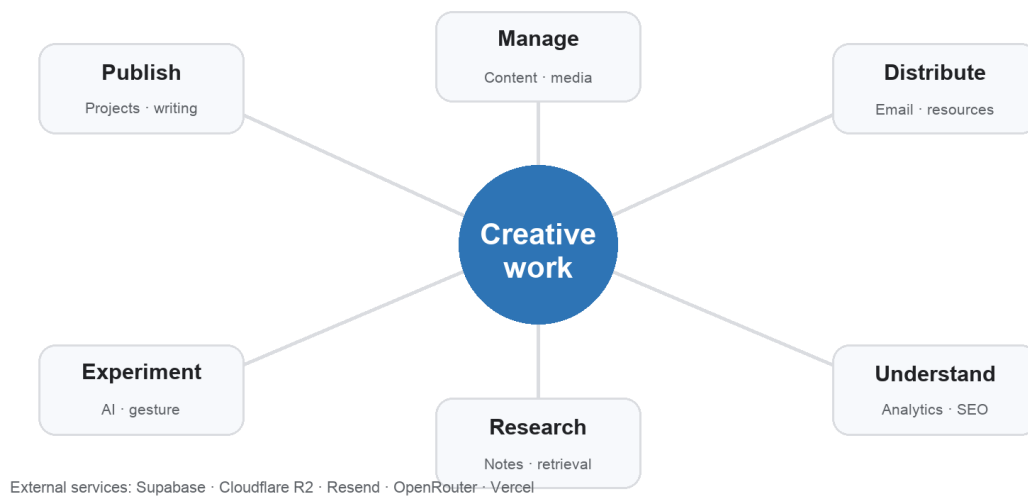
One creative practice. Too many disconnected tools.

Publishing, files, newsletters, analytics, research notes, and enquiries lived in separate systems. Every hand-off lost context. Repeated inbound questions suggested that this was not only a personal workflow issue.

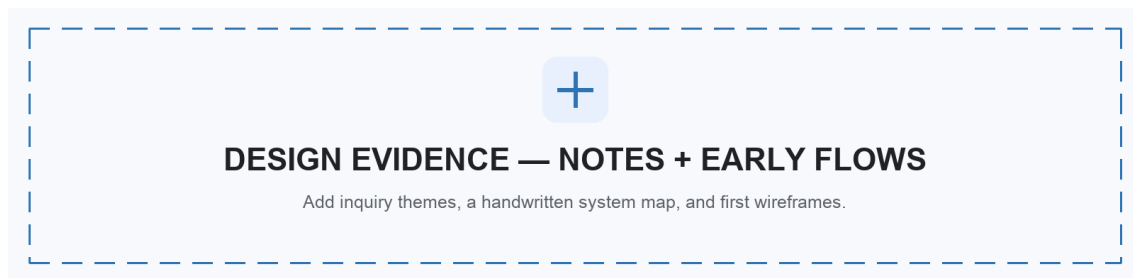
USER SIGNAL Repeated enquiries about projects, services, workshops, resources, and collaboration.

CORE NEED A single place to create, manage, publish, communicate, and learn.

PRODUCT HYPOTHESIS A modular operations platform can reduce context switching while preserving creative flexibility.



The product grew outward from one central object: the work itself.



Replace this placeholder with evidence, not decoration.

Research boundary: current evidence combines direct workflow observation and recurring enquiry patterns. The next phase requires structured interviews, competitor analysis, and task-based usability testing.

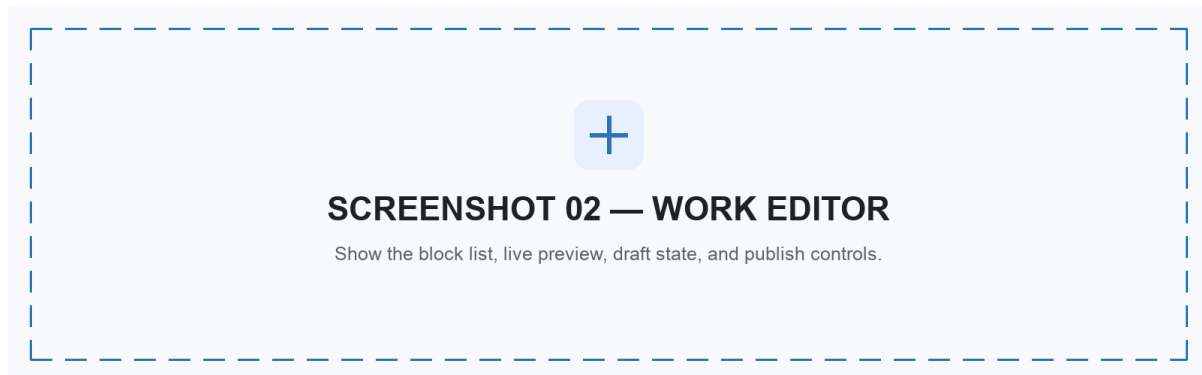
02 • CORE PRODUCT FLOW

Design the entire content lifecycle.

The key design decision was to treat publishing as a stateful workflow—not a save button. Every project moves through creation, preview, validation, publication, delivery, and learning.



Primary workflow • Each step exposes clear status, feedback, and recovery.



Show one task, one decision, and one visible system state.

```
await requirePortfolioAdmin();
const { data, error } = await supabase.rpc("portfolio_save_draft", {
  p_project_id: projectId,
  p_expected_lock_version: draft.lockVersion,
  p_payload: toSavePayload(draft),
});
```

The editor delegates authorization and conflict protection to a database function. A stale tab cannot overwrite newer work.

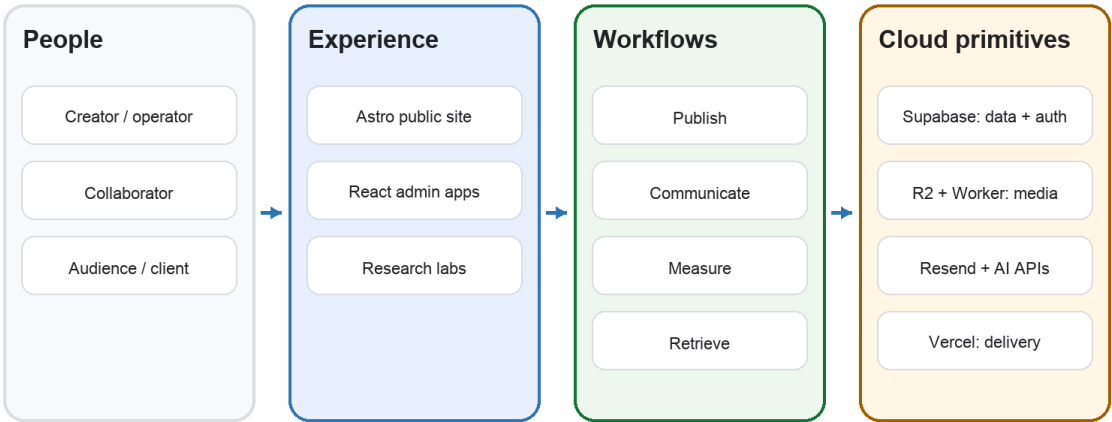
- **Recovery by design.** Published versions are immutable backups; restoring one never changes the live project until it is published again.
- **Safe public surface.** Sanitized views expose only published fields; draft and admin data remain private.

03 · PLATFORM ARCHITECTURE

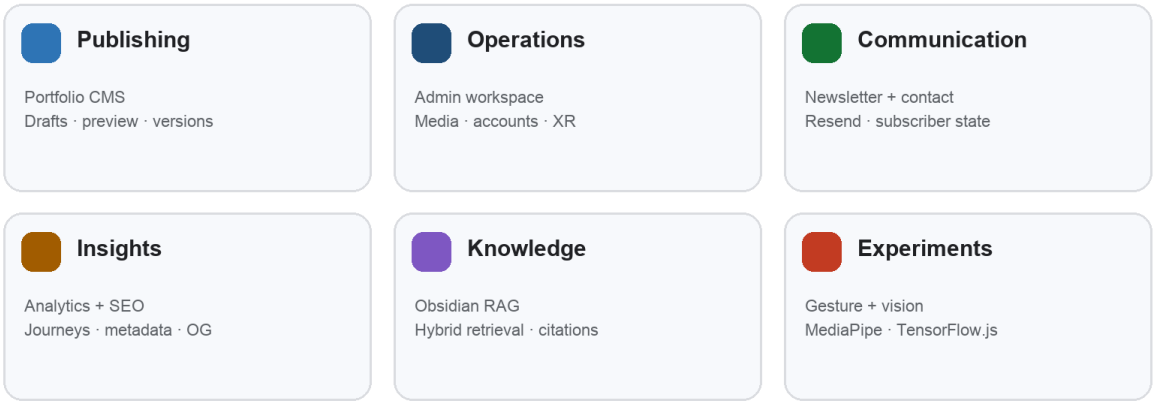
Show tasks. Hide infrastructure complexity.

Users interact with clear workflows. The platform coordinates data, identity, object storage, background processing, email, AI models, and delivery behind those workflows.

A modular platform: people see tasks, while the system coordinates services.



System model · Experience, workflow, and cloud layers remain modular.



Six product modules share identity, media, content, and system feedback.

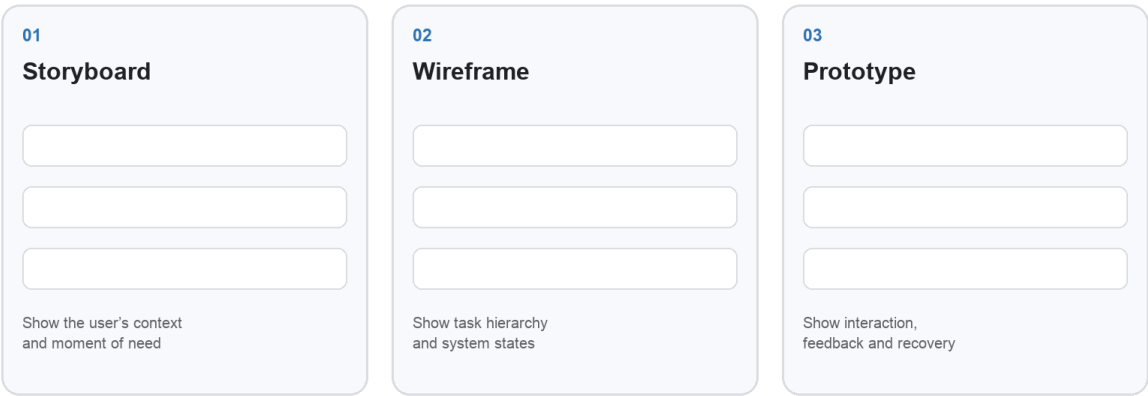
```
const questionEmbedding = await createEmbedding(searchQuery);
const { data } = await supabase.rpc("match_obsidian_chunks", {
  query_embedding: questionEmbedding,
  match_count: 12,
  match_threshold: 0.16,
  public_only: publicOnly,
});
```

The knowledge assistant retrieves only permitted note chunks before it generates a cited answer.

04 • INTERACTION DESIGN

Make complex systems feel calm.

The interface is organized around tasks and system states. Each screen should answer three questions: What can I do? What is happening? How can I recover?



Portfolio artifact placeholders · Replace with real storyboard, wireframe, and prototype frames.

- **Progressive disclosure.** Show essential controls first; reveal advanced settings when needed.
- **Consistent status language.** Draft, processing, published, failed, and recovered mean the same across tools.
- **Reversible actions.** Version history, protected media, and explicit publishing reduce destructive mistakes.
- **Accessible by default.** Semantic controls, keyboard access, alt text, reduced motion, and responsive layouts.
- **Shared patterns.** Layouts, blocks, tokens, validation, feedback, and empty states form the product language.

Enterprise patterns, clearly bounded

Capability	Working prototype	Enterprise validation
Access	Supabase Auth, RLS and server-only privileged keys	Organization tenancy, SSO and audit logs
Reliability	Immutable publish backups, queue retries and recovery	SLOs, incident runbooks and disaster recovery
Data	Public/private views and signed media uploads	Retention, governance and regional policy
Quality	Focused unit and end-to-end tests	CI gates, load tests and formal accessibility audit

Positioning: enterprise-pattern product prototype—not a claim of enterprise deployment or scale.

05 · REFLECTION

A working product—and a clearer design practice.

The prototype proves that publishing, communication, media, analytics, knowledge, and experimentation can share one coherent system. It also exposed where exploration must become product discipline.

MISTAKE I over-normalized the first portfolio schema.

LESSON I learned that a portable document snapshot can be safer and easier to author than maximum relational purity.

MISTAKE I let experiments grow into large components.

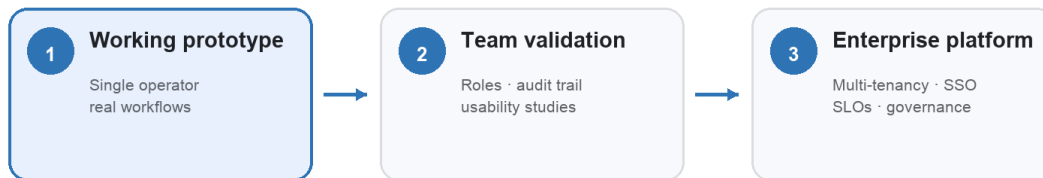
LESSON I learned to extract state machines, hooks, primitives, and design tokens once an idea proves useful.

MISTAKE Quality gates arrived too late.

LESSON I learned that tests, type checks, linting, builds, security checks, and documentation must run as one system.

MISTAKE Publishing exposed operational gaps.

LESSON I learned that cache freshness, staging access, observability, and recovery are part of the user experience.



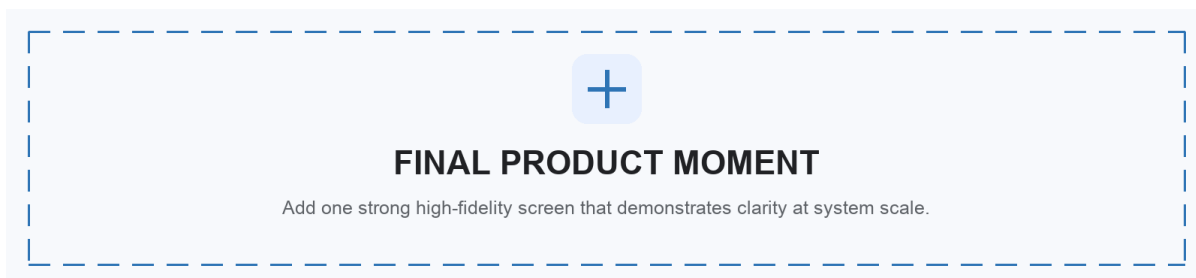
Scale is a validation path, not a visual claim.

Why this aligns with Google ACI UX

- Translates complex technical systems into understandable user flows.
- Connects storyboards, wireframes, prototypes, specifications, and implementation.
- Balances user needs with security, reliability, storage, compute, and operational constraints.
- Demonstrates end-to-end ownership: concept, system design, build, measurement, and iteration.

ROLE FIT

The domain is creative operations. The transferable skill is designing clarity, control, and trust across a complex cloud-backed system.



Closing visual placeholder · Add product link or QR code when the case study is published.